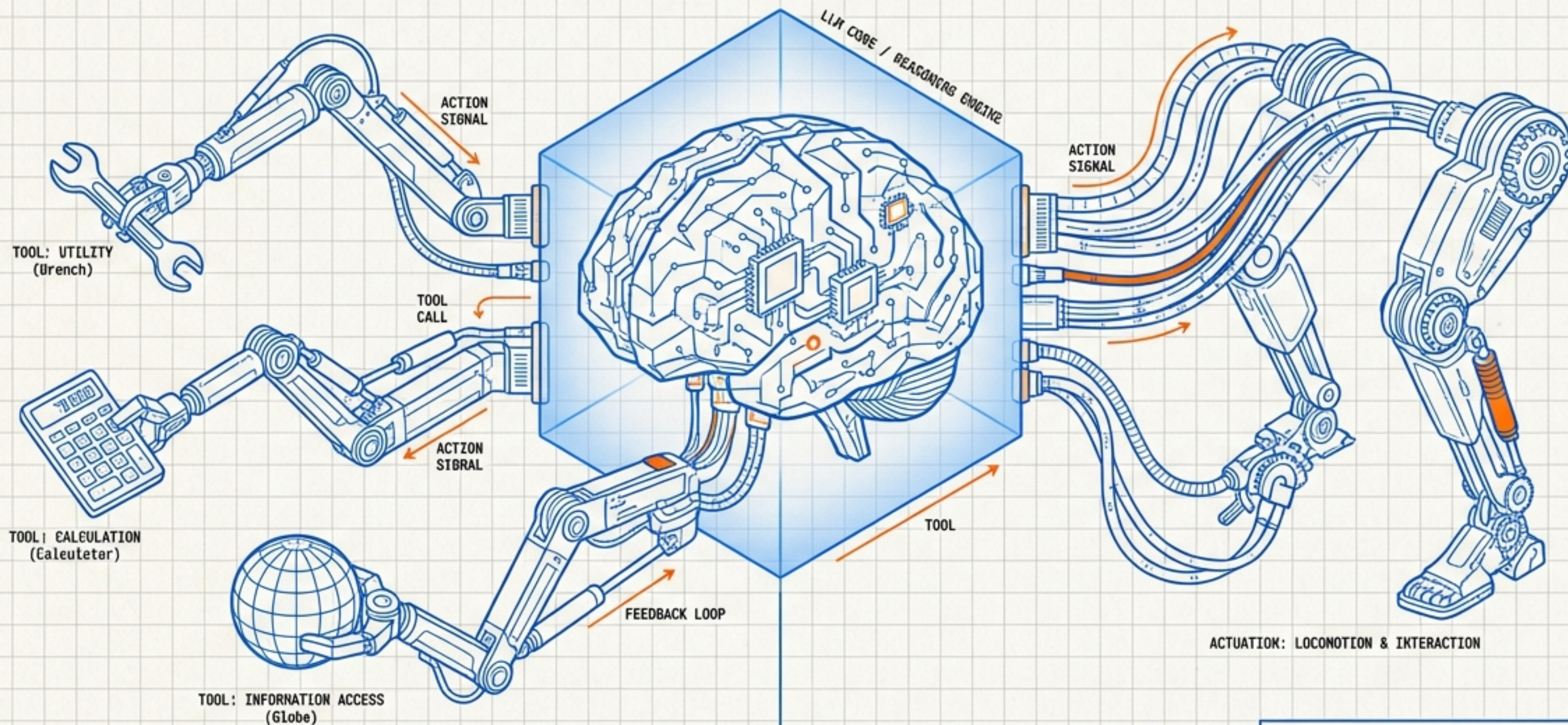


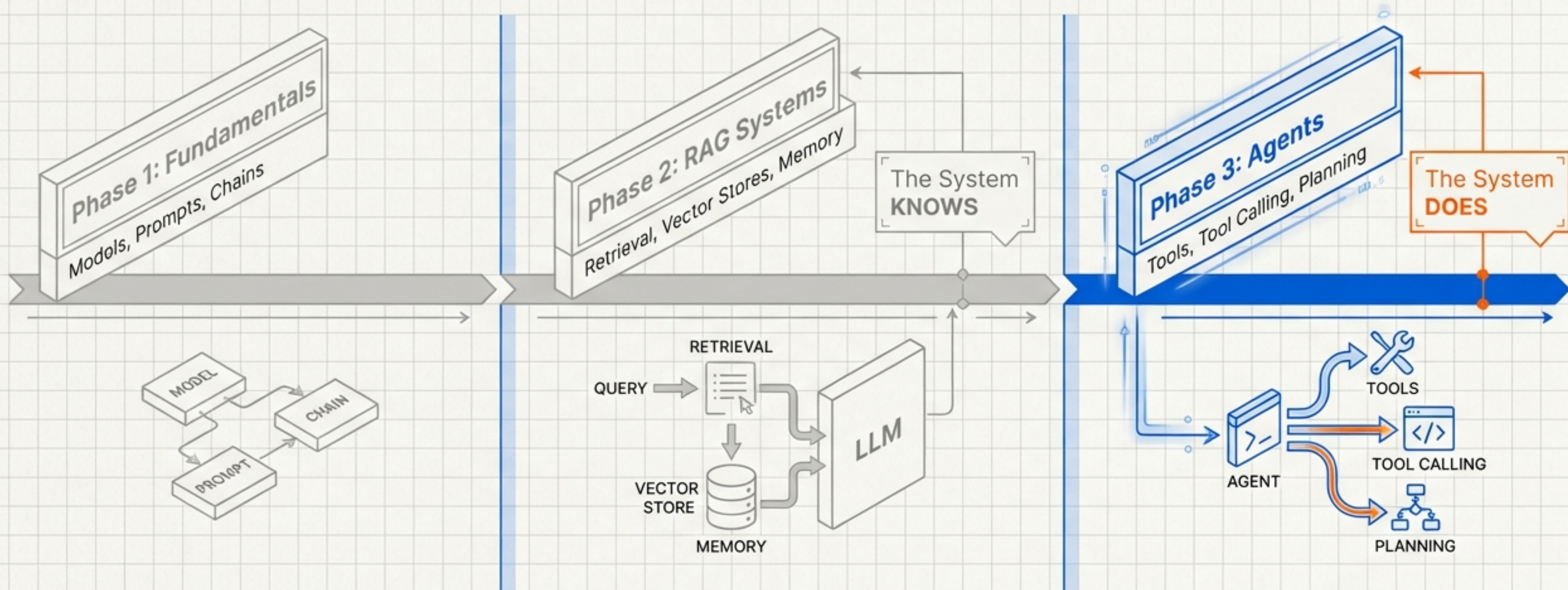
Giving LLMs Hands & Legs

A Guide to LangChain Tools:
From Passive Reasoning to Active Agents



CONCEPT: THE BRIDGE BETWEEN 'THINKING' AND 'ACTING'

The Evolution of LLM Systems



We have mastered **RAG (Retrieval Augmented Generation)**, giving the AI memory.
Now we enter the phase of **Agency**, giving the AI the ability to interact with the world.

The 'Brain in a Jar' Problem



The Superpowers

Reasoning: Can break down complex questions (Thinking).

Generation: Can articulate answers word-by-word (Speaking).



The Limitations

Travel: Can suggest a route, but cannot book the ticket.

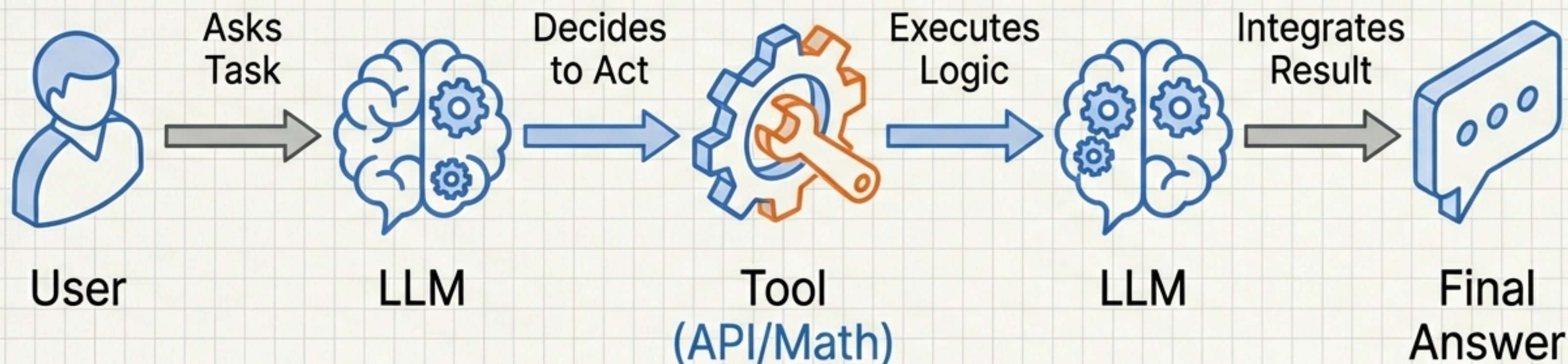
Math: Can do arithmetic, but fails at reliable complex calculation.

Connectivity: Cannot fetch live weather, post to Social Media, or query private SQL.

Insight: An LLM is a human body with a brain and mouth, but no limbs. It cannot execute the tasks it plans.

What is a Tool?

A Python function packaged so an LLM can understand **when** to call it and **how** to use it.



“Tools provide the mechanism to connect the ‘Brain’ to the external world.”

Level 1: LangChain Built-in Tools

Pre-built, production-ready 'prosthetics' for common use cases.



DuckDuckGo Search

Real-time web access
(overcoming knowledge cut-offs).



Wikipedia

Summarised topic queries and
background info.



Python REPL

Executing raw code for precise
maths (fixing hallucination).



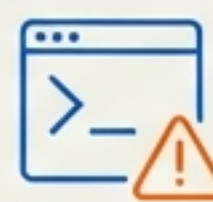
ArXiv / PubMed

Academic research paper
queries.



Gmail / Slack

Communication and messaging
integration.



Shell Tool

System command execution
(**Advanced**).

Implementing Built-ins: DuckDuckGo Example

```
from langchain_community.tools import DuckDuckGoSearchRun

# 1. Instantiate the tool
search_tool = DuckDuckGoSearchRun()

# 2. Tools are Runnable (they have .invoke)
result = search_tool.invoke("Current IPL news")

print(result)
```

Synchronous
Execution



Terminal Output

Virat Kohli is the all-time leading run-getter...

CSK IPL player qualification scenarios with four points from eight matches...

Power and Risk: The Shell Tool

Allows the LLM to execute terminal commands on the host machine. Requires

`langchain-experimental`.

Terminal Output

```
> shell_tool.invoke("whoami")  
root
```

```
> shell_tool.invoke("ls")  
[sample_data, project_files, config.json]
```



SAFETY ALERT: Giving an AI terminal access includes the ability to **modify or delete files** (e.g., ``rm -rf``). Use **with extreme caution** in production environments.

Level 2: Why Build Custom Tools?

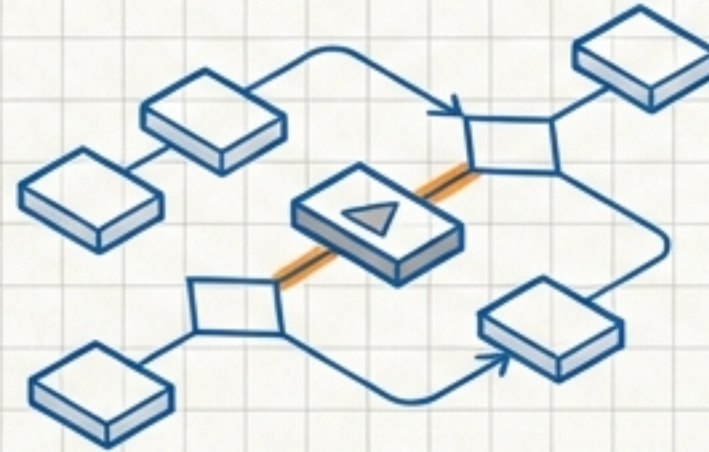
When off-the-shelf prosthetics don't fit the mission.

Private Data



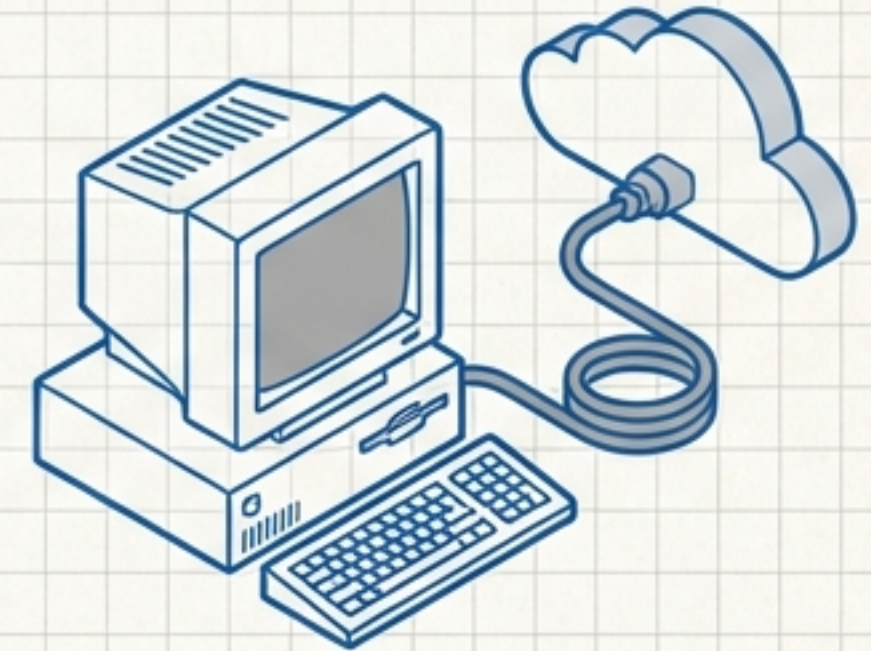
Connecting to internal company APIs, SQL databases, or CRMs.

Business Logic



Proprietary calculations or workflows unique to your application.

Legacy Systems



Interacting with older infrastructure not supported by public libraries.

Method A: The Decorator (@tool`)

The quickest way to turn a Python function into a LangChain Tool.

```
@tool  
def multiply(a: int, b: int) -> int:  
    """Multiplies two numbers."""  
    return a * b
```

1. The **Decorator**: Registers function as a Tool.

2. **Type Hints**: Define inputs for the Schema.

3. **Docstring**: The Instruction Manual for the LLM.

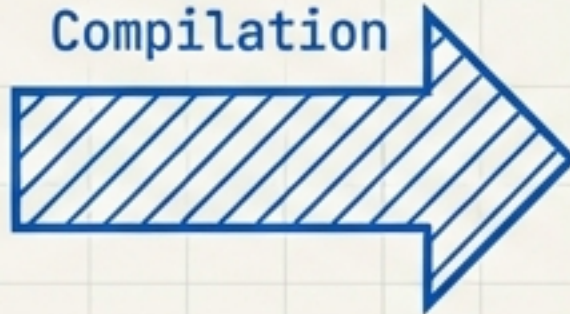
What the LLM Actually Sees

The LLM does not read Python code. It reads a JSON Schema generated from your function.

Python Function

```
def multiply(a: int, b: int) -> int:
    """Multiplies two numbers."""
    return a * b
```

LangChain
Compilation



JSON Schema Object

```
{
  "name": "multiply",
  "description": "Multiplies two numbers.",
  "parameters": {
    "type": "object",
    "properties": {
      "a": {"type": "integer"},
      "b": {"type": "integer"}
    },
    "required": ["a", "b"]
  }
}
```

Crucial: Your docstring becomes the '**description**' field. It is the UI for the Artificial Intelligence.

Method B: StructuredTool (Pydantic)

For strict validation and complex input schemas.

```
from langchain_core.tools import StructuredTool
from pydantic import BaseModel, Field
```

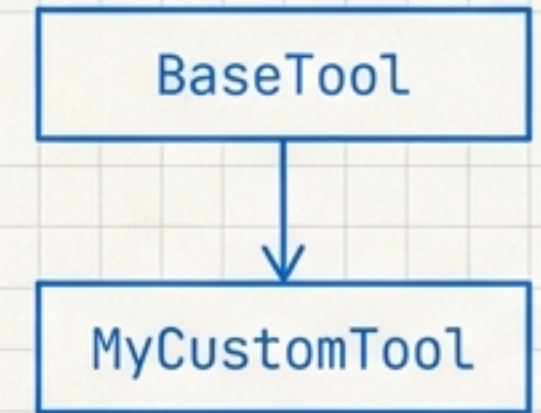
```
class MultiplyInput(BaseModel):
    a: int = Field(description="First number")
    b: int = Field(description="Second number")
```

```
tool = StructuredTool.from_function(
    func=multiply,
    args_schema=MultiplyInput
)
```

Enforces rigid types.
If LLM sends a string,
Pydantic rejects it
automatically.

Method C: Subclassing BaseTool

The Industrial Grade Solution.



```
from langchain_core.tools import BaseTool

class MultiplyTool(BaseTool):
    name = "multiply"
    description = "Multiplies two numbers"
    args_schema: Type[BaseModel] = MultiplyInput
```

```
    def _run(self, a: int, b: int) ->
```

```
    int:
        return a * b
```

```
    async def _arun(self, a: int, b: int) -> int:
        # Async implementation (Not possible with @tool)
        return a * b
```

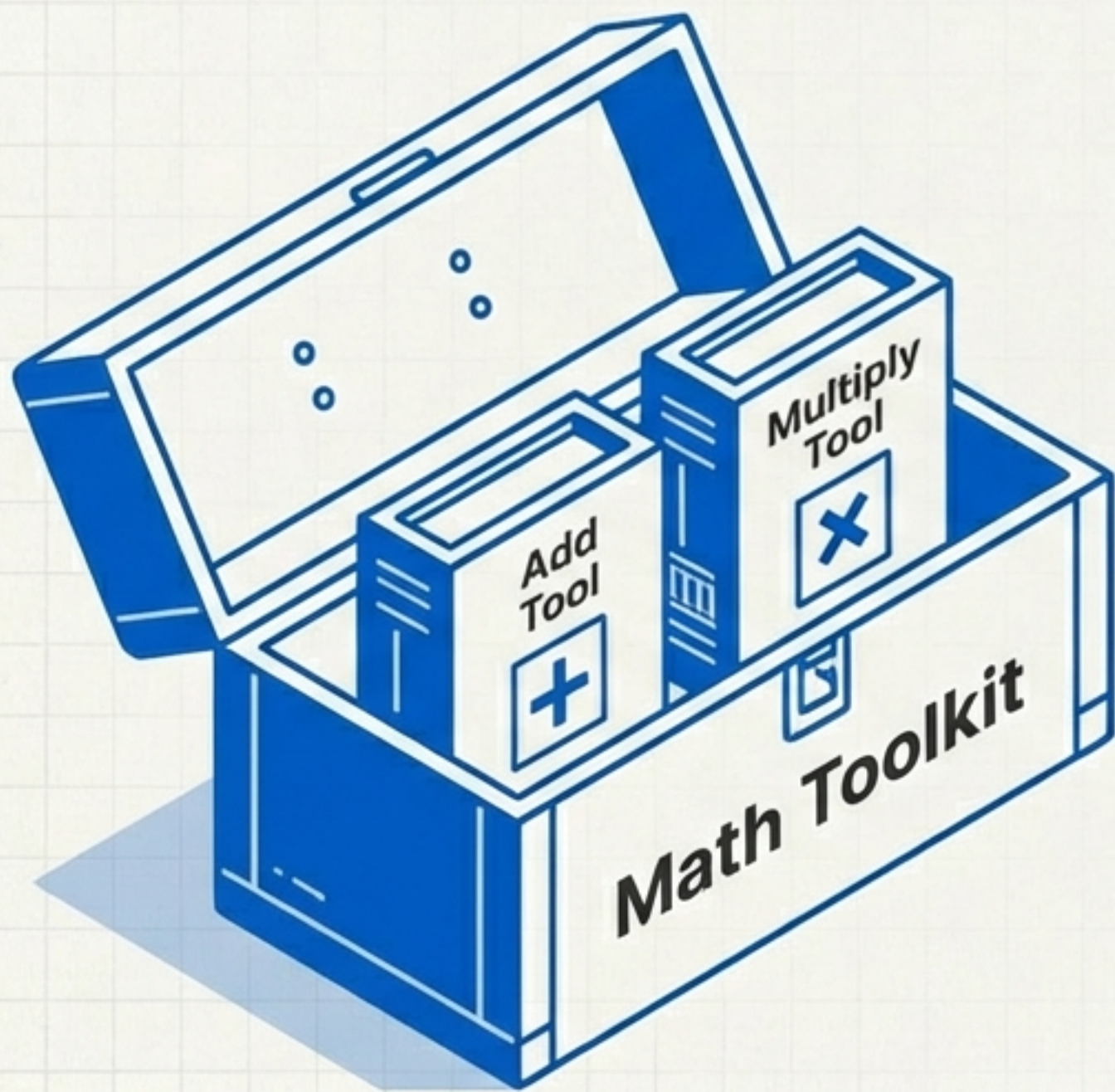
- 🔧 Maximal Control over state and validation.
- 📄 Supports Asynchronous execution (`_arun`).
- 🔧 The bedrock of all LangChain tools.

Choosing the Construction Method

<code>@tool</code> Decorator	Best for prototyping, simple functions, and speed. Covers 90% of use cases.	 Start Here
<code>StructuredTool</code>	Best for complex input schemas requiring requiring strict Pydantic validation.	
<code>BaseTool</code> <code>Subclass</code>	Best for “Industrial Grade” apps requiring Async support or persistent state.	

Scaling Up: Toolkits

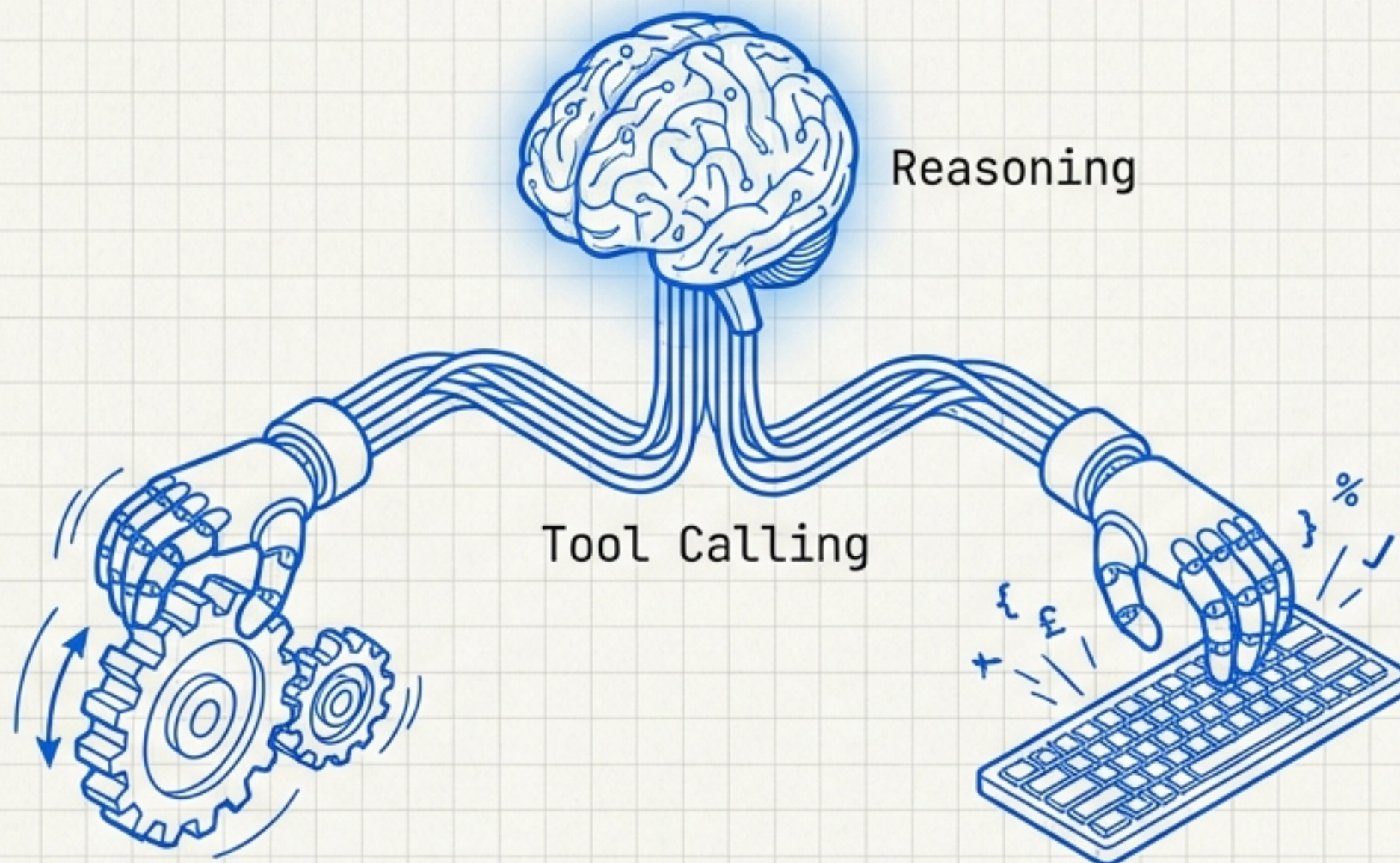
A “Toolkit” is a collection of related tools packaged for reusability.



```
class MathToolkit:  
    def get_tools(self):  
        return [add_tool, multiply_tool]
```

Benefit: Write once, reuse everywhere.
Plug the entire toolkit into any Agent.

The Complete Picture: Towards Agency



LLMs = Brain (Reasoning)
Tools = Hands (Execution)
Agent = The System

**NEXT UP:
TOOL CALLING.**
Teaching the brain
how to use the hands.